Enter Spring

Copy "test_spring" from template projects n import it as existing Maven Project
Force Update the Maven project , to download the dependencies.

Topics :
Why Spring
What is it ?
Dependency Injection / IoC
Spring Bean life cycle
Bean Wiring

Proxy: The Proxy Design Pattern is a structural design pattern that provides a surrogate or placeholder for another object to control access to it. This pattern is used to create a representative object that controls access to another object, which can be useful in various situations such as controlling access to a resource, lazy initialization, logging, and more.

Why Spring ?
1. Simplifies overall java development
2. Allows the developers to create loosely coupled applications.
Manages the dependencies , so that dependent objects are completely de coupled from the dependecies.
3. Supplies ready made implementation of design patterns (singleton , factory , proxy , MVC , front controller n many more)
Spring supports the MVC pattern for developing monolithic web application as well as RESTful web services.
4. Reduces Boilerplate code.
5. Helps to build Easy, Simple, and Lightweight applications
6. Excellent support for JDBC as well as ORM , along with automatic transaction management support
7. Modular and non intrusive design
8. Provides smooth integration with other frameworks such as Hibernate , Struts , EJB.
9. Supports AOP (Aspect oriented programming) for the separation n modularization of cross cutting concerns (=repetative tasks) from the core tasks
(eg - request handling logic : controller bean , B.L - service bean, DAO - data access logic)
10. Supports Unit testing as well as integration testing
11. Securing web apps as well as RESTful web services becomes easy
n many more....

What is it ?
1. container --manages life cycle of spring beans
(spring bean --- java obj whose life cycle completely managed by SC(spring container)
eg :REST controller, controller, service,DAO.
2. Framework of frameworks -
framework --It provides ready made implementation of standard design patterns(eg :MVC,Proxy,singleton,factory, ORM ...)
It's a modular framework.
Overview of Spring frmwork modules
refer to diagram - "Spring Reference Material\diags\spring-framework-modules.png"

Compare the layers : Refer to whiteboard diagram

Spring is modular n extensive framework.

Why Spring : loosely coupled application
Via : D.I / AOP

What is dependency injection(D.I) ?

In the earlier programming -

Layers used : JSP|Servlet---Java Bean ---DAO(Utils) -- POJO --DB
Dependent Objs -- JavaBean , Hibernate based DAO, JDBC Based DAO
Dependencies --- DAO,HibernateUtils(SessionFactory) , DBUtils(Fixed DB connection)

All of these are the examples of tight coupling.

Why --Any time the nature of the dependency changes , dependent obj is affected(i.e you will have to make changes in dependent obj as well !)

eg : When the dependency of Java Bean changes from JDBC Based DAO to Hibernate based DAO , in case of user authentication , javabean class has to be modified to handle invalid login case(i.e handle NoResultException)

Tight coupling is strongly un desirable , since it's difficult to maintain or extend the application.

Why it was leading to tight coupling ?
Dependent objects were managing their dependencies , in this traditional|conventional programming model.

eg : Java bean(dependent) creates the instance of DAO(dependency)

In above examples , Java bean creates the instance of DAO.
Hibernate based DAO , gets SF from HibUtils.
JDBC based DAO ,gets db connection from DBUtils.

What is D.I ?(Dependency injection=wiring=collaboration between dependent & dependency)
It is a design pattern that tries to decouple dependent objects from their dependencies.
Instead of creating their own dependencies internally, dependent objects receive the dependencies from the 3rd party (eg Spring Container) directly at run time.

This approach leads to loose coupling ,which leads further to maintanable code .

In the conventional programming , the control of managing dependencies was with dependent objects.
Now onwards , since dependent objects are no longer managing dependencies --its called as IoC ---Inversion of control

Hollywood principle states - You don't call us , we will call you....

SC tells these dependent objs - not to manage their dependencies , since it will be automatically provided by SC(Spring Container)

eg : UserController
@Autowired
private IUserService service;

In DAO layer
@AutoWired
private SessionFactory sf;

Pre requisite : Already added STS plug-in / STS 3.9.18
Steps for spring nature to Java project

Important : Extract spring api-docs
Objective : Create Spring based Java SE project

1. Create Maven based Java SE project with spring dependencies
eg - test_spring
2. Create dependent n dependency classes
4. Refer : <resources> & create spring bean config xml file.(Using STS 3.9.x support)
5. Add namespace <beans>

To Configure SC using xml based meta data instructions
Add <bean> tags in the xml file

More details about <bean> tag
Important Attributes of <bean> tag
1. id -mandatory attribute - refers to bean unique id
2. class --- mandatory -- Fully qualified bean class name
3. scope ---optional .
Default scope = singleton
In a standalone main based application --- singleton | prototype
In web app - singleton | prototype | request | session | global session (exists ONLY under portlet based environment)
Meaning -
singleton --- Spring Container(SC) will share single bean instance for multiple requests/demands(via ctx.getBean)
prototype -- SC creates NEW bean instance per request/demand.

4. lazy-init --- boolean attribute. default value=false.
Applicable only to singleton beans.
By default - SC will auto create singleton spring bean instance --- @ SC start up.

5. init-method --to specify the name of custom init method
(default pattern -- public void anyName() throws Exception{init logic})
- It will be auto called by SC after D.I
Called for singleton as well as prototype beans.

6. destroy-method --to specify the name of custom destroy method
(default pattern -- public void anyName() throws Exception{..})
- It will be auto called by SC before GC of spring bean (applicable only to singleton beans)

API
How to get ready to use spring bean from SC ?
API of BeanFactory
public <T> T getBean(String beanId,Class<T> beanClass) throws BeansException
T - type of the spring bean

Spring bean life cycle
Types of wiring in explicit mode - setter Based D.I , Constructor based D.I , Factory Based D.I

Types of wiring in implicit (auto wiring) mode - setter Based D.I , Constructor based D.I only.
In auto wiring , XML tags reduce BUT there is no reduction in Java code
(i.e setters | parameterized construct) is still required.